

BahaAI — IoT Hardware Assembly Guide

From loose components to live readings on the dashboard

Flood Early-Warning System · Municipality of Consolacion, Cebu
Companion to the **IoT Hardware Buying Guide** · Prepared 13 September 2026

START HERE

This guide assumes your parts have arrived and the BahaAI web app already runs (`docker compose up` → `http://localhost:8000`). It takes you from an empty breadboard to a physical sensor that makes the dashboard react.

Work in order. Sections 2 to 4 form a chain of test gates. Each gate proves one thing works before the next is added. Wiring everything at once and powering up is how students lose days guessing which of nine new things is broken.

THE ONE MISTAKE THAT RUINS EVERYTHING

The ultrasonic sensor measures **downward** — the air gap between itself and the water surface. The app wants **water level**, measured upward from the channel bed. Your firmware must convert between them:

```
water_level = MOUNT_HEIGHT - measured_distance
```

Get this backwards and every alert fires inverted: the dashboard reports a critical flood when the channel is dry, and reports safe as water reaches the sensor. Section 6 covers measuring `MOUNT_HEIGHT` correctly.

1 Before you start

Fifteen minutes of setup now saves a day of confusion later.

1.1 Parts needed for this stage

Only **Set A** from the buying guide is needed to reach a live reading. Set B parts (relay, pump, servo, enclosure) are not touched until Section 7.

- ESP32 development board (30-pin WROOM-32) and a USB **data** cable matching its socket — **USB-C** on current devkits, micro-USB on older ones. Check the board before buying
- HC-SR04 ultrasonic for bench work; JSN-SR04T / AJ-SR04M for the real build
- DHT22 temperature and humidity sensor
- 830-point breadboard, jumper wires (male-male and male-female)
- Resistors: $1 \times 1 \text{ k}\Omega$, $1 \times 2 \text{ k}\Omega$, $1 \times 10 \text{ k}\Omega$ — from the assortment kit
- 5 V 2 A USB adapter

CABLE WARNING — THE SINGLE MOST COMMON FAILURE

Many USB cables are **charge-only**: they carry power (VBUS, GND) but not data (D+, D-). The ESP32's red PWR LED lights normally and the board looks alive, yet it never appears as a COM port and the IDE's Port menu stays greyed out. Cables bundled with **chargers, power banks and LED lights** are the usual offenders; phone sync cables are almost always fine.

A lit LED does not clear the cable — power and data are separate wires in the same jacket. Test a cable independently by plugging it into a phone: if the PC offers file transfer, the data lines work.

USB-C BOARDS: USE AN A-TO-C CABLE, NOT C-TO-C

Many ESP32 devkits with a USB-C socket omit the two 5.1 kΩ **CC pull-down resistors** the USB-C specification requires. A USB-C host then cannot detect the board as a sink, so a **C-to-C cable may deliver nothing at all** — no power, no enumeration — while the identical board works perfectly on a **USB-A to USB-C** cable, which uses legacy signalling and ignores CC entirely.

Buy **USB-A to USB-C** and plug into a normal rectangular USB-A port. If your laptop has only USB-C ports, use a powered USB-A hub or dock rather than fighting this.

1.2 Software to install on the laptop

1 **Arduino IDE 2.x** — download from arduino.cc/en/software and install.

2 **ESP32 board support.** Open **File** → **Preferences**. In *Additional boards manager URLs*, paste:

```
https://espressif.github.io/arduino-esp32/package_esp32_index.json
```

Then **Tools** → **Board** → **Boards Manager**, search `esp32`, and install **esp32 by Espressif Systems**. It is a large download — let it finish completely.

3 **Libraries.** **Tools** → **Manage Libraries**, then install **DHT sensor library** by Adafruit. When the IDE offers to also install **Adafruit Unified Sensor** as a dependency, accept.

No JSON library is required — the firmware in Section 5 builds its payload as a plain string.

4 **USB-to-serial driver.** Windows 11 does **not** reliably ship the WCH drivers, despite what many tutorials claim. Install both from wch-ic.com — they cover different chips and do not conflict:

- **CH341SER** — CH340 / CH341
- **CH343SER** — CH9102 / CH343, common on current devkits

Extract the archive first, then **right-click** → **Run as administrator**. Run from inside the zip or without admin and the installer reports success while staging nothing. These pre-stage into the driver store, so installing with no board attached is fine — Windows binds automatically when a matching chip appears.

IF NO COM PORT APPEARS, ANSWER THIS QUESTION BEFORE TOUCHING DRIVERS

Ask first: **does Windows see the device at all?** The two failure modes look identical in the Arduino IDE — a greyed-out Port menu — but have completely different fixes.

In Device Manager	Meaning
Device listed with a yellow triangle , VID_1A86 or VID_10C4 visible	Driver missing. Install the package above.
Nothing appears at all when you plug in	Not a driver problem. A device with no driver still enumerates — Windows reads its VID/PID before any driver loads. Total silence means the USB data lines are not connected: charge-only cable, dead port, or faulty board.

A lit red PWR LED does not clear the cable. USB power (VBUS/GND) and USB data (D+/D-) are separate wires; a charge-only cable lights the board and enumerates nothing. **Charger-bundled cables are the usual culprit.** Test a cable independently by plugging it into a phone — if the PC does not offer file transfer, that cable can never program an ESP32.

5 **Board settings.** Plug in the ESP32, then set **Tools** → **Board** → **ESP32 Arduino** → **ESP32 Dev Module** and **Tools** → **Port** → the COM port that appeared.

1.3 Network requirements — read carefully

Requirement	Why it matters
-------------	----------------

2.4 GHz Wi-Fi only	The ESP32 has no 5 GHz radio. If your router broadcasts one name for both bands, the ESP32 may never associate. Use a dedicated 2.4 GHz SSID, or a phone hotspot set to 2.4 GHz.
Same network as the laptop	The ESP32 posts directly to your laptop. Both must be on the same subnet. A phone hotspot with both devices joined is the most reliable setup for a defense.
Never <code>localhost</code>	<code>http://localhost:8000</code> means "this device" — to the ESP32, that is the ESP32 itself. You must use the laptop's LAN IP, for example <code>http://192.168.1.14:8000</code> . Gate 5 covers finding it.
Firewall opened	Windows blocks inbound port 8000 by default, so the app is reachable from the laptop but invisible to the ESP32. Gate 5 covers opening it.

2 Server preparation — do this first

Three changes to the Laravel app. The first is a security fix you cannot skip; the other two unlock hardware features the database already supports.

WHY THIS COMES BEFORE THE WIRING

Change 2.1 adds a device token. Your firmware needs that token baked in, so mint it now — otherwise you will write the firmware twice.

2.1 Authenticate the reading endpoint REQUIRED

Today `POST /api/sensors/{sensor}/readings` has no middleware at all. Anyone who can reach the address can inject a fake three-metre reading and fire genuine critical alerts at genuine residents. This is acceptable on an isolated bench network and indefensible anywhere else.

The approach below gives **each sensor its own token**, so a compromised node can only post as itself — stronger than one shared key, and a good paragraph for your paper.

database/migrations/2026_09_13_000001_add_api_token_to_sensors_table.php — NEW FILE

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::table('sensors', function (Blueprint $table) {
            $table->string('api_token', 64)->nullable()->unique()->after('status');
        });
    }

    public function down(): void
    {
        Schema::table('sensors', function (Blueprint $table) {
            $table->dropColumn('api_token');
        });
    }
};
```

Add `api_token` to the model's fillable list, and hide it so it never leaks into an Inertia page prop:

app/Models/Sensor.php — EDIT

```
#[Fillable([
    'flood_zone_id', 'name', 'type', 'latitude', 'longitude',
    'status', 'battery_level', 'last_reading_at', 'api_token',
])]
class Sensor extends Model
{
    /** @use HasFactory<SensorFactory> */
    use HasFactory;

    protected $hidden = ['api_token'];
```

The middleware compares the presented bearer token against the hash stored for the sensor in the URL. `hash_equals` is used rather than `===` to avoid leaking information through response timing.

app/Http/Middleware/EnsureSensorToken.php — NEW FILE

```
<?php

namespace App\Http\Middleware;

use App\Models\Sensor;
use Closure;
use Illuminate\Http\Request;
use Symfony\Component\HttpFoundation\Response;

class EnsureSensorToken
{
    public function handle(Request $request, Closure $next): Response
    {
        $sensor = $request->route('sensor');

        if (! $sensor instanceof Sensor) {
            $sensor = Sensor::find($sensor);
        }

        $presented = $request->bearerToken();

        if (! $sensor || ! $sensor->api_token || ! $presented
            || ! hash_equals($sensor->api_token, hash('sha256', $presented))) {
            abort(401, 'Invalid or missing device token.');
```

Register the alias alongside the two that already exist:

bootstrap/app.php — EDIT

```
use App\Http\Middleware\EnsureSensorToken;

$middleware->alias([
    'admin' => EnsureUserIsAdmin::class,
    'staff' => EnsureUserIsStaff::class,
    'sensor.token' => EnsureSensorToken::class,
]);
```

Apply it to the route:

routes/api.php — EDIT

```
Route::post('/sensors/{sensor}/readings', [SensorDataController::class, 'store'])
    ->middleware('sensor.token')
    ->name('api.sensor-readings.store');
```

Finally, a command to mint tokens — written in the same attribute style as the existing commands in `app/Console/Commands` :

app/Console/Commands/IssueSensorToken.php — NEW FILE

```
<?php

namespace App\Console\Commands;

use App\Models\Sensor;
use Illuminate\Console\Attributes\Description;
use Illuminate\Console\Attributes\Signature;
use Illuminate\Console\Command;
use Illuminate\Support\Str;

#[Signature('sensors:token {sensor : The sensor ID}')]
#[Description('Issue a new API token for a physical sensor node')]
class IssueSensorToken extends Command
{
    public function handle(): int
    {
        $sensor = Sensor::find($this->argument('sensor'));

        if (! $sensor) {
            $this->error('Sensor not found. ');

            return self::FAILURE;
        }

        $plain = Str::random(48);
        $sensor->update(['api_token' => hash('sha256', $plain)]);

        $this->info("Token for sensor #{ $sensor->id } ( { $sensor->name } ):");
        $this->line($plain);
        $this->warn('Copy it now – it is stored hashed and cannot be shown again. ');

        return self::SUCCESS;
    }
}
```

EXISTING TESTS WILL FAIL — AND THAT IS CORRECT

The suite in `tests/` posts readings without a token, so those cases now return 401. Update them to mint a token and send an `Authorization` header. A reviewer who sees the tests updated alongside the fix reads it as deliberate; a reviewer who sees a skipped test does not.

2.2 Accept battery level from the device RECOMMENDED

The `sensors` table has a `battery_level` column and the dashboard displays it, but the reading endpoint rejects the field — so a real node has no way to report it. Battery level belongs to the *sensor*, not to an individual reading, so it is split out of the payload before the reading row is created.

[app/Http/Requests/Api/StoreSensorReadingRequest.php](#) — EDIT

```
public function rules(): array
{
    return [
        'water_level' => ['required', 'numeric', 'min:0'],
        'rainfall' => ['nullable', 'numeric', 'min:0'],
        'temperature' => ['nullable', 'numeric'],
        'humidity' => ['nullable', 'numeric', 'between:0,100'],
        'battery_level' => ['nullable', 'numeric', 'between:0,100'],
        'recorded_at' => ['nullable', 'date'],
    ];
}
```

DO NOT SKIP THE CONTROLLER CHANGE

The controller currently spreads `...$request->validated()` straight into `SensorReading::create()`. Add `battery_level` to the rules without the change below and every POST will fail — `sensor_readings` has no such column.

app/Http/Controllers/Api/SensorDataController.php — EDIT

```
public function store(StoreSensorReadingRequest $request, Sensor $sensor, AlertService $alertService):
JsonResponse
{
    $data = $request->validated();
    $battery = $data['battery_level'] ?? null;
    unset($data['battery_level']);

    $reading = SensorReading::create([
        'sensor_id' => $sensor->id,
        ...$data,
        'recorded_at' => $data['recorded_at'] ?? now(),
    ]);

    $attributes = ['last_reading_at' => $reading->recorded_at];

    if ($battery !== null) {
        $attributes['battery_level'] = $battery;
    }

    $sensor->update($attributes);

    $sensor->loadMissing('floodZone');
    $alert = $alertService->evaluateReading($sensor, $reading);

    return response()->json([
        'message' => 'Reading recorded successfully.',
        'reading' => $reading,
        'alert' => $alert,
    ], 201);
}
```

2.3 Let actuator hardware read its own state NEEDED FOR SECTION 7

The Smart Actuation screen writes pump, siren and floodgate state into the database, but nothing exposes that state outside the web UI. A relay board has no way to find out it should switch on. This read-only endpoint closes the loop; the node polls it every 30 seconds.

app/Http/Controllers/Api/ActuatorStateController.php — NEW FILE

```
<?php

namespace App\Http\Controllers\Api;

use App\Http\Controllers\Controller;
use App\Models\ActuatorDevice;
use Illuminate\Http\JsonResponse;
use Illuminate\Http\Request;

class ActuatorStateController extends Controller
{
    public function index(Request $request): JsonResponse
    {
        $devices = ActuatorDevice::query()
            ->when($request->integer('zone'), fn ($query, $zone) => $query->where('flood_zone_id', $zone))
            ->orderBy('id')
            ->get(['id', 'name', 'type', 'is_on', 'mode', 'status']);

        return response()->json(['devices' => $devices]);
    }
}
```

routes/api.php — [EDIT](#)

```
Route::get('/actuators', [ActuatorStateController::class, 'index']->name('api.actuators.index'));
```

BE HONEST ABOUT THIS ONE IN THE PAPER

This endpoint is unauthenticated. It is read-only and exposes no personal data, so the risk is far lower than the reading endpoint's — but it does let anyone on the network see device states. State the limitation and note the mitigation (same per-device token scheme as 2.1) as future work, rather than leaving a reviewer to find it.

2.4 Apply and verify

```
docker compose exec app php artisan migrate
docker compose exec app php artisan sensors:token 1
```

Copy the printed token somewhere safe — it is stored hashed and cannot be retrieved again. If you lose it, run the command again to issue a replacement.

To find the sensor ID, sign in as admin and open **Admin** → **Sensors**; the ID is in the row and in the edit URL. Pick a sensor whose flood zone you are happy to trigger alerts for.

CHECKPOINT

Migration applied, token in hand, and `php artisan test` still green after updating the API tests. Now you can pick up a soldering iron.

3 Wiring the sensor node

One pin map, one voltage divider, and a strict rule about power. Get these right and the firmware just works.

3.1 Pin map

These pins avoid the ESP32's strapping pins (GPIO 0, 5, 12, 15) and the flash pins (GPIO 6–11), which cause boot failures that look like random faults.

Component	Component pin	ESP32 pin	Notes
Ultrasonic HC-SR04 / JSN-SR04T	VCC	VIN (5V)	Needs 5 V. Will not work reliably from the 3V3 pin.
	TRIG	GPIO 17	Direct. A 3.3 V pulse triggers it fine.
	ECHO	GPIO 16	Through the voltage divider in 3.2 — never direct.
	GND	GND	
DHT22	VCC (pin 1)	3V3	Powered at 3.3 V so its data line is already 3.3 V logic — no divider needed.
	DATA (pin 2)	GPIO 4	10 k Ω pull-up resistor from DATA to 3V3. Bare 4-pin sensors need this; small breakout boards often have it built in.
	GND (pin 4)	GND	Pin 3 is not connected.
Status LED	—	GPIO 2	Onboard blue LED on most devkits. No wiring needed.
Battery sense (Section 8)	18650 +	GPIO 34	Through a 100 k Ω / 100 k Ω divider. Input-only ADC1 pin — correct choice for analogue reads.
Relay board (Section 7)	IN1 pump	GPIO 26	Set B only. Relay VCC to 5 V, GND common with the ESP32.
	IN2 siren	GPIO 27	
	IN3 floodgate	GPIO 14	
	IN4 spare	GPIO 13	
SG90 servo (Section 7)	Signal (orange)	GPIO 25	Red to 5 V, brown to GND. Never power a servo from the 3V3 pin.

3.2 The voltage divider — the part that kills boards

ESP32 PINS ARE NOT 5 V TOLERANT

The ultrasonic module runs at 5 V, so its ECHO pin outputs **5 V**. ESP32 GPIOs are rated to 3.3 V, absolute maximum 3.6 V. Wiring ECHO straight to a GPIO may work for an hour, a week, or not at all — then the pin dies silently and you debug firmware for two days. Fit the divider.

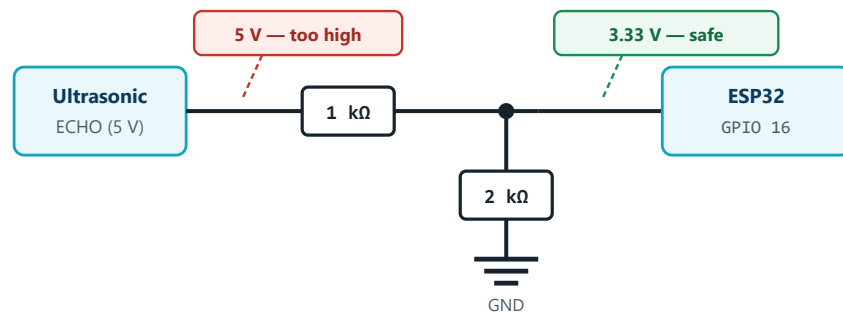


Figure 1 — Divide ECHO down before it reaches the ESP32. $5\text{ V} \times 2\text{ k}\Omega \div (1\text{ k}\Omega + 2\text{ k}\Omega) = 3.33\text{ V}$.

Build it on the breadboard: ECHO into one row, a 1 kΩ resistor from that row to a second row, a 2 kΩ resistor from the second row down to the ground rail, and a jumper from the second row to GPIO 16. If you only have 2.2 kΩ, that gives 3.44 V — still within the absolute maximum and acceptable. TRIG needs no divider because the ESP32 drives it, not the other way round.

3.3 Power rules

Rule	Reason
Use the 5 V 2 A adapter , not a laptop USB port, once the pump or servo is connected	A laptop port supplies about 500 mA. The pump alone draws roughly 200 mA and the SG90 can spike near 700 mA when stalled. The ESP32 browns out and reboots, which looks exactly like a firmware crash.
Every ground must be common	The ESP32, ultrasonic, relay board, servo and pump supply all share one ground. Separate grounds give floating references and wildly erratic readings.
Never power a motor or pump from an ESP32 pin	GPIO pins source about 40 mA maximum. The relay switches a separate supply; the GPIO only drives the relay's input.
Power the DHT22 from 3V3	Keeps its data line at 3.3 V logic, so no level shifting is required.

3.4 Wiring order

1. **Unplug the ESP32.** Never rewire a powered board.
2. Seat the ESP32 across the breadboard's centre channel, straddling both halves.
3. Run the rails first: a jumper from **GND** to the blue rail, and from **3V3** to the red rail. Colour discipline here prevents most wiring errors.
4. Wire the ultrasonic: VCC to **VIN**, GND to the blue rail, TRIG to GPIO 17.
5. Build the divider from 3.2, then jumper its midpoint to GPIO 16.
6. Wire the DHT22: VCC to the red rail, GND to the blue rail, DATA to GPIO 4, and the 10 kΩ pull-up from DATA to the red rail.
7. **Check every connection twice**, especially that no 5 V wire reaches a GPIO directly. Then plug in.

4 Bring-up — nine gates

Each gate adds exactly one capability and ends with an observable result. Do not start the next gate until the current one passes. When something breaks, it is almost always in the gate you just added.

GATE 1 The board talks to the IDE

Prove the toolchain, cable and drivers work before any sensor is involved.

```
void setup() {
  Serial.begin(115200);
  pinMode(2, OUTPUT);
}

void loop() {
  digitalWrite(2, HIGH);
  Serial.println("BahaAI node alive");
  delay(500);
  digitalWrite(2, LOW);
  delay(500);
}
```

Upload, then open **Tools** → **Serial Monitor** and set the baud rate to **115200**.

If upload fails with "Failed to connect": hold the BOOT button while the IDE prints "Connecting...", release once it starts writing. Some clone boards need this every time.

Pass when: the onboard LED blinks once per second and `BahaAI node alive` repeats in the Serial Monitor.

GATE 2 Distance on the serial monitor

Use the cheap HC-SR04 here. Save the waterproof JSN-SR04T until the code is proven.

```
#define PIN_TRIG 17
#define PIN_ECHO 16

void setup() {
  Serial.begin(115200);
  pinMode(PIN_TRIG, OUTPUT);
  pinMode(PIN_ECHO, INPUT);
}

void loop() {
  digitalWrite(PIN_TRIG, LOW);
  delayMicroseconds(4);
  digitalWrite(PIN_TRIG, HIGH);
  delayMicroseconds(10);
  digitalWrite(PIN_TRIG, LOW);

  unsigned long duration = pulseIn(PIN_ECHO, HIGH, 30000UL);

  if (duration == 0) {
    Serial.println("no echo (out of range, or check wiring)");
  } else {
    Serial.printf("%.1f cm\n", duration / 58.0);
  }
  delay(500);
}
```

Hold a book at measured distances and compare against a tape measure. Readings should track within a centimetre or two.

Always "no echo"? Confirm VCC is on `VIN` and not `3V3`, confirm the divider midpoint reaches GPIO 16, and confirm grounds are common. On a **JSN-SR04T v3.0**, also check that the `R27` pad is empty — a resistor there selects a serial mode that ignores TRIG entirely.

Pass when: printed centimetres track a tape measure as you move a flat object.

GATE 3 Temperature and humidity

```
#include <DHT.h>
#define PIN_DHT 4
DHT dht(PIN_DHT, DHT22);

void setup() {
  Serial.begin(115200);
  dht.begin();
}

void loop() {
  float t = dht.readTemperature();
  float h = dht.readHumidity();

  if (isnan(t) || isnan(h)) {
    Serial.println("DHT read failed — check pull-up and wiring");
  } else {
    Serial.printf("%.1f C  %.1f %%\n", t, h);
  }
  delay(2500);
}
```

The DHT22 refuses reads closer together than about two seconds, so the 2.5-second delay is deliberate. Persistent `nan` almost always means a missing 10 kΩ pull-up, or a blue DHT11 declared as a DHT22 — change the type to `DHT11` if that is what arrived.

Pass when: a plausible room temperature and humidity print every 2.5 seconds. Breathe on the sensor — humidity should climb, then settle.

GATE 4 Wi-Fi association

```
#include <WiFi.h>

const char* WIFI_SSID = "YourNetwork";
const char* WIFI_PASSWORD = "YourPassword";

void setup() {
  Serial.begin(115200);
  delay(200);
  WiFi.mode(WIFI_STA);
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

  Serial.print("Connecting");
  while (WiFi.status() != WL_CONNECTED) {
    delay(400);
    Serial.print(".");
  }
  Serial.println();
  Serial.print("Connected. ESP32 IP: ");
  Serial.println(WiFi.localIP());
}

void loop() {}
```

Stuck printing dots? In order of likelihood: the network is 5 GHz; the SSID or password has a typo (both are case-sensitive); the hotspot has hit its device limit. Note the printed IP — it confirms which subnet the ESP32 joined, which must match the laptop's.

Pass when: an IP address prints, on the same subnet as the laptop (first three numbers match).

GATE 5 A reading from the laptop, by hand

Prove the server accepts a reading before asking the ESP32 to produce one. This separates network problems from firmware problems permanently.

Find the laptop's LAN IP. In PowerShell run `ipconfig` and read the *IPv4 Address* under the adapter actually in use — the Wi-Fi one, not Ethernet, and not a `172.x` Docker or WSL adapter.

Open the firewall once, in an **Administrator** PowerShell:

```
netsh advfirewall firewall add rule name="BahaAI 8000" dir=in action=allow protocol=TCP localport=8000
```

Post a reading. Replace the IP, sensor ID and token:

```
curl.exe -X POST http://192.168.1.14:8000/api/sensors/1/readings ^
-H "Content-Type: application/json" ^
-H "Accept: application/json" ^
-H "Authorization: Bearer YOUR_TOKEN_HERE" ^
-d "{\"water_level\":1.85,\"temperature\":27.4,\"humidity\":82}"
```

POWERSHELL TRAP

In PowerShell, `curl` is an alias for `Invoke-WebRequest`, which takes completely different arguments. Type `curl.exe` — with the extension — or the command will fail in a confusing way.

A `201` response with the created reading means success. Interpreting the failures:

Response	Meaning
401	Token wrong, or missing for that sensor. Re-issue with <code>sensors:token</code> .
404	That sensor ID does not exist. Check Admin → Sensors .
422	Validation failed — the response body names the field. Usually <code>water_level</code> missing or negative.
Timeout	Firewall, wrong IP, or the app is not running.

Then run the same command from a **phone browser or another device on the same network** pointed at that URL — if the laptop can reach it but the phone cannot, it is the firewall, not the app.

Pass when: you get `201`, and the new reading appears on the sensor's page under **Water Levels** in the dashboard.

GATE 6 The ESP32 posts a fixed number

Same request, now from the board — with a hardcoded water level so nothing depends on the sensor yet.

```
#include <WiFi.h>
#include <HTTPClient.h>

const char* WIFI_SSID      = "YourNetwork";
const char* WIFI_PASSWORD  = "YourPassword";
const char* SERVER_HOST    = "192.168.1.14";
const int   SERVER_PORT    = 8000;
const int   SENSOR_ID      = 1;
const char* DEVICE_TOKEN   = "YOUR_TOKEN_HERE";

void setup() {
  Serial.begin(115200);
  WiFi.mode(WIFI_STA);
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
  while (WiFi.status() != WL_CONNECTED) { delay(400); Serial.print("."); }
  Serial.println("\nWi-Fi up");

  String url = "http://" + String(SERVER_HOST) + ":" + String(SERVER_PORT)
    + "/api/sensors/" + String(SENSOR_ID) + "/readings";

  HTTPClient http;
  http.begin(url);
  http.addHeader("Content-Type", "application/json");
  http.addHeader("Accept", "application/json");
  http.addHeader("Authorization", String("Bearer ") + DEVICE_TOKEN);

  int code = http.POST("{\"water_level\":1.85}");
  Serial.printf("HTTP %d\n", code);
  Serial.println(http.getString());
  http.end();
}

void loop() {}
```

A negative HTTP code is a client-side failure — the request never left the board. `-1` means it could not connect: wrong IP, firewall, or the ESP32 landed on a different subnet than the laptop.

Pass when: the Serial Monitor prints `HTTP 201` and a fresh reading appears in the dashboard.

GATE 7 Real measurements, correctly converted

Load the full firmware from Section 5. Set `MOUNT_HEIGHT_M` using the method in Section 6, and drop `POST_INTERVAL_MS` to `15000` (15 seconds) while testing so you are not waiting five minutes per attempt.

Watch the Serial Monitor as you raise and lower a flat object beneath the sensor. **Distance and water level must move in opposite directions.** If they move together, `MOUNT_HEIGHT_M` is wrong or the subtraction is inverted.

RESTORE THE INTERVAL BEFORE THE DEMO

Leave it at 15 seconds and you generate 240 readings an hour, flooding the charts and slowing the prediction query. Set it back to `5UL * 60UL * 1000UL` once calibration is done.

Pass when: the dashboard water level rises as the object rises, and a reading arrives on every interval.

GATE 8 Make a flood happen

This is the moment worth rehearsing for the panel. Mount the JSN-SR04T over the test tank and pour water in.

1. Fit the waterproof sensor to the tank lid or a crossbar, facing straight down at the water.
2. **Leave at least 25 cm of air** between the sensor face and the highest water — it is blind closer than that.
3. Re-measure `MOUNT_HEIGHT_M` for the new mounting and re-flash.
4. Set that zone's thresholds to suit the tank in **Admin** → **Flood Zones** — see Section 6.3.
5. Pour slowly and watch the dashboard cross **safe** → **warning** → **critical**.

Each threshold crossing creates one alert and dispatches notifications. Alerts are deduplicated per sensor and severity, so a second warning-level reading will *not* create a second warning alert while the first is still active — that is intended behaviour, not a bug. Resolve the alert to see it fire again.

Pass when: an alert appears on the Alerts page with the right severity, and notification log entries exist for that zone's residents.

GATE 9 Prediction from real data

The prediction fits a linear regression over the last 120 minutes and needs at least three readings in that window. At a five-minute interval you have 24 — comfortable.

To demonstrate a rising trend without waiting two hours, pour water in gradually over 15–20 minutes so several readings capture the rise, then force a prediction:

```
docker compose exec app php artisan predictions:generate
```

A steady pour produces a high R^2 , so confidence will be high and *time to critical* plausible. A single sudden dump produces a poor fit and low confidence — which is itself a good thing to show a panel, because it demonstrates the confidence score doing its job.

Pass when: the Predictions page shows a positive rate of rise, a time-to-critical estimate, and a confidence score from your own water.

5 The sensor node firmware

Complete and ready to flash. Edit only the block marked at the top, then upload.

Three things here are worth defending in your paper, because they are deliberate engineering rather than tutorial copy-paste:

- **Median-of-seven filtering.** A single ultrasonic ping off a rippling water surface is noisy and occasionally wildly wrong. Taking the median of seven samples discards outliers without smearing genuine level changes the way an average would.
- **Temperature-compensated speed of sound.** The usual `duration / 58` shortcut assumes a fixed air temperature. Since a DHT22 is already fitted, the firmware computes `c = 331.3 + 0.606 × T` instead. Between a 22 °C night and a 35 °C afternoon this is roughly a 2 % correction — about 2 cm at a 1 m range.
- **Range gating.** Readings inside the 25 cm blind zone or beyond 4.5 m are rejected rather than reported, so a failed ping cannot masquerade as a water level.

`firmware/baha_sensor_node/baha_sensor_node.ino` — NEW FILE

```

/* =====
 * BahaAI – Flood Sensor Node
 * ESP32 + JSN-SR04T ultrasonic + DHT22
 *
 * Posts to POST /api/sensors/{id}/readings
 * Sends   water_level (m), temperature (C), humidity (%),
 *         battery_level (%)
 * ===== */

#include <WiFi.h>
#include <HTTPClient.h>
#include <DHT.h>

/* ----- EDIT EVERYTHING IN THIS BLOCK ----- */
const char* WIFI_SSID    = "YourNetwork";
const char* WIFI_PASSWORD = "YourPassword";

const char* SERVER_HOST   = "192.168.1.14"; // laptop LAN IP – never "localhost"
const int   SERVER_PORT   = 8000;
const int   SENSOR_ID     = 1;             // from Admin -> Sensors
const char* DEVICE_TOKEN  = "paste-token-from-sensors:token";

// Vertical distance from the sensor face down to the channel bed, in metres.
// Measure it – do not estimate it. See Section 6.1.
const float MOUNT_HEIGHT_M = 1.20;

// 5 minutes. Offline is flagged at 30 min; predictions need 3 readings per 2 h.
const unsigned long POST_INTERVAL_MS = 5UL * 60UL * 1000UL;

const bool REPORT_BATTERY = false;        // set true once Section 8 is wired
/* ----- */

#define PIN_TRIG    17
#define PIN_ECHO    16
#define PIN_DHT     4
#define PIN_LED     2
#define PIN_BATTERY 34

DHT dht(PIN_DHT, DHT22);

const int   SAMPLE_COUNT = 7;             // median filter width
const float SENSOR_MIN_M = 0.25;          // JSN-SR04T blind zone
const float SENSOR_MAX_M = 4.50;
const float BATT_EMPTY_V = 3.00;
const float BATT_FULL_V  = 4.20;

unsigned long lastPost = 0;
bool firstRun = true;

void setup() {
  Serial.begin(115200);
  delay(200);

  pinMode(PIN_TRIG, OUTPUT);
  pinMode(PIN_ECHO, INPUT);
  pinMode(PIN_LED, OUTPUT);
  digitalWrite(PIN_TRIG, LOW);

  dht.begin();
  connectWifi();
}

void connectWifi() {
  if (WiFi.status() == WL_CONNECTED) return;

  WiFi.mode(WIFI_STA);
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

```

```

Serial.print("Wi-Fi connecting");

unsigned long started = millis();
while (WiFi.status() != WL_CONNECTED && millis() - started < 20000UL) {
    delay(400);
    Serial.print(".");
}
Serial.println();

if (WiFi.status() == WL_CONNECTED) {
    Serial.print("Connected, IP ");
    Serial.println(WiFi.localIP());
} else {
    Serial.println("Wi-Fi failed – will retry next cycle");
}
}

/* One ping. Returns metres, or -1 if the echo is missing or out of range. */
float readDistanceOnce(float tempC) {
    digitalWrite(PIN_TRIG, LOW);
    delayMicroseconds(4);
    digitalWrite(PIN_TRIG, HIGH);
    delayMicroseconds(10);
    digitalWrite(PIN_TRIG, LOW);

    unsigned long duration = pulseIn(PIN_ECHO, HIGH, 30000UL);
    if (duration == 0) return -1.0;

    // Speed of sound varies with air temperature; the DHT22 lets us correct it.
    float speed = 331.3 + 0.606 * tempC; // m/s
    float metres = (duration * 1e-6 * speed) / 2.0; // halve: out and back

    if (metres < SENSOR_MIN_M || metres > SENSOR_MAX_M) return -1.0;
    return metres;
}

/* Median of several pings – rejects the outliers a rippling surface produces. */
float readDistanceMedian(float tempC) {
    float samples[SAMPLE_COUNT];
    int valid = 0;

    for (int i = 0; i < SAMPLE_COUNT; i++) {
        float d = readDistanceOnce(tempC);
        if (d > 0) samples[valid++] = d;
        delay(100); // let echoes die away between pings
    }

    if (valid < 3) return -1.0; // not enough good data to trust

    for (int i = 1; i < valid; i++) { // insertion sort
        float key = samples[i];
        int j = i - 1;
        while (j >= 0 && samples[j] > key) { samples[j + 1] = samples[j]; j--; }
        samples[j + 1] = key;
    }

    return samples[valid / 2];
}

float readBatteryPercent() {
    uint32_t total = 0;
    for (int i = 0; i < 8; i++) total += analogReadMilliVolts(PIN_BATTERY);

    float volts = (total / 8.0f) * 2.0f / 1000.0f; // undo the 1:2 divider
    float pct = (volts - BATT_EMPTY_V) / (BATT_FULL_V - BATT_EMPTY_V) * 100.0f;

    return constrain(pct, 0.0f, 100.0f);
}

```

```

bool postReading(float level, float tempC, float humidity, float battery) {
    String url = "http://" + String(SERVER_HOST) + ":" + String(SERVER_PORT)
        + "/api/sensors/" + String(SENSOR_ID) + "/readings";

    String body = "{\"water_level\":" + String(level, 2);
    if (!isnan(tempC))    body += ", \"temperature\":" + String(tempC, 2);
    if (!isnan(humidity)) body += ", \"humidity\":" + String(humidity, 2);
    if (battery >= 0)     body += ", \"battery_level\":" + String(battery, 1);
    body += "}";

    HTTPClient http;
    http.begin(url);
    http.setTimeout(10000);
    http.addHeader("Content-Type", "application/json");
    http.addHeader("Accept", "application/json");
    http.addHeader("Authorization", String("Bearer ") + DEVICE_TOKEN);

    int code = http.POST(body);
    Serial.printf("POST %s -> HTTP %d\n", body.c_str(), code);

    if (code == 401) Serial.println(" Token rejected – re-issue with sensors:token");
    if (code == 404) Serial.println(" No sensor with that ID");
    if (code == 422) Serial.println(http.getString());
    if (code < 0)   Serial.println(" Server unreachable – check IP and firewall");

    http.end();
    return code == 201;
}

void loop() {
    if (!firstRun && millis() - lastPost < POST_INTERVAL_MS) return;

    firstRun = false;
    lastPost = millis();

    digitalWrite(PIN_LED, HIGH);
    connectWifi();

    float tempC    = dht.readTemperature();
    float humidity = dht.readHumidity();
    float forSpeed = isnan(tempC) ? 26.0 : tempC;    // Cebu annual average fallback

    float distance = readDistanceMedian(forSpeed);

    if (distance < 0) {
        Serial.println("No usable echo – skipping this cycle");
        digitalWrite(PIN_LED, LOW);
        return;
    }

    // The sensor measures DOWN; the app wants UP.
    float level = MOUNT_HEIGHT_M - distance;
    if (level < 0) level = 0;

    float battery = REPORT_BATTERY ? readBatteryPercent() : -1.0;

    Serial.printf("distance %.3f m | level %.3f m | %.1f C | %.1f %%\n",
        distance, level, tempC, humidity);

    if (WiFi.status() == WL_CONNECTED) {
        postReading(level, tempC, humidity, battery);
    } else {
        Serial.println("Offline – reading discarded");
    }
}

```

```
digitalWrite(PIN_LED, LOW);  
}
```

NOTE ON RAINFALL

The firmware deliberately never sends `rainfall`. Section 4 of the buying guide explains why: the P75 module detects wet or dry, it does not measure millimetres, and publishing its output as a millimetre value would be inventing data. Leave the field empty and let the OpenWeatherMap integration supply rainfall — or build the tipping bucket and send real millimetres.

READINGS ARE DISCARDED, NOT QUEUED

If Wi-Fi is down at post time, that cycle's reading is lost. For a bench demonstration this is fine and keeps the code readable. For real deployment you would buffer to flash and replay on reconnect — worth naming as future work, since `recorded_at` is already accepted by the API precisely so a device can backfill delayed readings.

6 Calibration and mounting

The firmware is only as good as the one number you measure by hand.

6.1 Measuring MOUNT_HEIGHT the reliable way

Rather than fighting a tape measure in an awkward spot, let the sensor measure it for you. With the sensor fixed in its **final** position and the tank or channel **empty**:

1. Flash the firmware with any value in `MOUNT_HEIGHT_M` — it does not matter yet.
2. Open the Serial Monitor and read the printed `distance` value.
3. With no water present, that distance **is** the mount height. Copy it into `MOUNT_HEIGHT_M` and re-flash.

This cancels out mounting tilt and the offset between the board and the transducer face, both of which a tape measure gets wrong. Re-do it whenever the sensor is moved, even slightly.

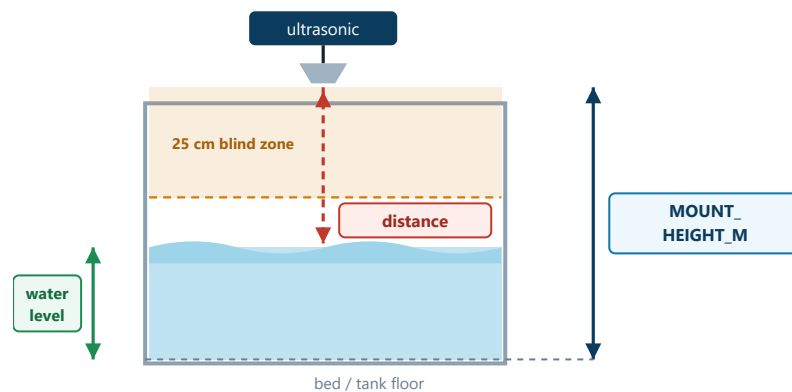


Figure 2 — The sensor reports the red measurement. The app needs the green one. The firmware subtracts.

6.2 Verify the inversion before trusting anything

With the tank part-filled, confirm the two numbers move in opposite directions. If they move together, the subtraction is backwards and every alert will be inverted.

State	distance	water_level	Verdict
Tank empty	0.60 m	0.00 m	Correct
Half full	0.42 m	0.18 m	Correct — one up, one down
Near full	0.30 m	0.30 m	Correct
If water level falls as you pour water in, <code>MOUNT_HEIGHT_M</code> is wrong or the sign is flipped. Stop and fix it now.			

6.3 Scaling the thresholds to a test tank

The seeded zones use field thresholds — warning at 1.5 m, critical at 3.0 m. A 40 cm tank will never reach those, so nothing will ever alert. Edit the zone your sensor belongs to in **Admin** → **Flood Zones** and scale the thresholds down.

Worked example for a sensor mounted **60 cm** above the tank floor:

Quantity	Value	Why
<code>MOUNT_HEIGHT_M</code>	0.60 m	Measured by the method in 6.1.
Highest measurable water	0.35 m	60 cm minus the 25 cm blind zone. Water above this is invisible to the sensor.

safe_threshold	0.00	Baseline.
warning_threshold	0.15	15 cm of water — easy to reach with a jug.
critical_threshold	0.28	Leaves 7 cm of headroom below the blind zone, so critical is reachable without the sensor going blind mid-demonstration.

WRITE THIS DOWN FOR THE PANEL

Say plainly that thresholds were scaled for bench demonstration and that field values remain 1.5 m and 3.0 m. Scaling is normal engineering practice; a reviewer who discovers it unannounced may read it as the system being tuned to pass its own test.

6.4 What degrades an ultrasonic reading

Cause	Effect and mitigation
Tilt	The beam reflects away from the sensor instead of back. Mount within about 5° of vertical — check with a phone spirit level. This is the most common cause of a sensor that reads intermittently.
Beam spread	The cone is roughly 15° wide — about 26 cm across at 1 m range. Anything inside that cone (tank wall, pipe, bracket, cable) reflects sooner than the water and reports as a falsely high level. Keep the cone clear.
Ripples and turbulence	Scatter the echo. The median filter absorbs most of it. Mount over still water rather than a fast chute where possible.
Foam and floating debris	Reflect at the debris surface, not the water. Unavoidable outdoors — mention it as a known limitation.
Air temperature	Already corrected in firmware using the DHT22. Without correction, expect roughly 2 % error across a normal day.
Condensation on the face	Can block the transducer. In the field enclosure, point the sensor down and add a small drip edge.

7 The actuator node — Set B

Pump, siren and floodgate driven by the Smart Actuation screen. Requires change 2.3 from Section 2.

LOW-VOLTAGE DC ONLY

The relays on these boards are printed with mains ratings. **Do not switch mains voltage in this project.** Breadboard wiring, exposed terminals and a student enclosure provide none of the isolation, strain relief or clearance that mains work requires, and a flooded demonstration involves water on the same table. Switch only the 3–6 V pump and the 5 V buzzer. Say so in the paper — it reads as competence, not timidity.

7.1 Wiring the relay board

- Relay `VCC` to the 5 V rail, `GND` to common ground.
- `IN1` → GPIO 26 (pump), `IN2` → GPIO 27 (siren), `IN3` → GPIO 14 (floodgate), `IN4` → GPIO 13 (spare).
- The switched device goes through the **COM** and **NO** (normally open) screw terminals, in series with its own supply — so the load is off until the relay closes.

TWO RELAY-BOARD QUIRKS THAT WASTE AFTERNOONS

Most boards are active LOW. `digitalWrite(pin, LOW)` turns the relay *on*. If your relays are on at boot and click off when commanded, this is why — the firmware below has a flag for it.

The JD-VCC jumper. If relays do not click at all, the 3.3 V logic is too weak for the opto-isolators. Remove the `VCC-JD-VCC` jumper and feed `JD-VCC` from 5 V separately, keeping grounds common.

7.2 Polling firmware

Install **ArduinoJson** (by Benoit Blanchon) from the Library Manager first. The node polls every 30 seconds and mirrors whatever the dashboard says.

ARDUINOJSON VERSION

The code below uses the version 7 API. If the Library Manager installed version 6, `JsonDocument doc;` will not compile — either upgrade to 7, or change that one line to `DynamicJsonDocument doc(2048);`. Everything else is identical between the two versions.

firmware/baha_actuator_node/baha_actuator_node.ino — NEW FILE


```

#include <WiFi.h>
#include <HTTPClient.h>
#include <ArduinoJson.h>

const char* WIFI_SSID      = "YourNetwork";
const char* WIFI_PASSWORD  = "YourPassword";
const char* SERVER_HOST    = "192.168.1.14";
const int   SERVER_PORT    = 8000;
const int   ZONE_ID        = 1;           // flood zone this node serves

#define RELAY_PUMP    26
#define RELAY_SIREN   27
#define RELAY_GATE    14

const bool RELAY_ACTIVE_LOW = true;      // most 4-channel boards are
const unsigned long POLL_INTERVAL_MS = 30UL * 1000UL;

unsigned long lastPoll = 0;

void setRelay(int pin, bool on) {
    digitalWrite(pin, (on == RELAY_ACTIVE_LOW) ? LOW : HIGH);
}

void setup() {
    Serial.begin(115200);

    int pins[] = { RELAY_PUMP, RELAY_SIREN, RELAY_GATE };
    for (int pin : pins) {
        pinMode(pin, OUTPUT);
        setRelay(pin, false);           // everything off at boot
    }

    WiFi.mode(WIFI_STA);
    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
    while (WiFi.status() != WL_CONNECTED) { delay(400); Serial.print("."); }
    Serial.println("\nWi-Fi up");
}

void pollActuators() {
    String url = "http://" + String(SERVER_HOST) + ":" + String(SERVER_PORT)
        + "/api/actuators?zone=" + String(ZONE_ID);

    HTTPClient http;
    http.begin(url);
    http.setTimeout(10000);
    http.addHeader("Accept", "application/json");

    int code = http.GET();
    if (code != 200) {
        Serial.printf("actuator poll -> HTTP %d\n", code);
        http.end();
        return;
    }

    JsonDocument doc;
    DeserializationError err = deserializeJson(doc, http.getString());
    http.end();

    if (err) {
        Serial.println("actuator JSON parse failed");
        return;
    }

    for (JsonObject device : doc["devices"].as<JsonArray>()) {
        const char* type = device["type"];
        bool on = device["is_on"];
    }
}

```

```

    if (strcmp(type, "pump") == 0) {
        setRelay(RELAY_PUMP, on);
    } else if (strcmp(type, "siren") == 0) {
        setRelay(RELAY_SIREN, on);
    } else if (strcmp(type, "floodgate") == 0 || strcmp(type, "valve") == 0) {
        setRelay(RELAY_GATE, on);
    }

    Serial.printf(" %-10s %s\n", type, on ? "ON" : "off");
}
}

void loop() {
    if (millis() - lastPoll < POLL_INTERVAL_MS) return;
    lastPoll = millis();

    if (WiFi.status() != WL_CONNECTED) {
        WiFi.reconnect();
        return;
    }

    pollActuators();
}

```

DEMONSTRATION TIP

Thirty seconds is a long silence in front of a panel. Drop `POLL_INTERVAL_MS` to 5 seconds for the demonstration so the pump responds while the examiner is still looking at the screen, and say that the field value is higher to conserve battery.

7.3 The servo floodgate

A servo needs a timed pulse train, not a relay contact, so drive it directly from `GPIO 25` with the ESP32Servo library — red to the 5 V rail, brown to common ground, orange to the GPIO. Never power it from the 3V3 pin; its stall current will brown out the board.

```

#include <ESP32Servo.h>

Servo gate;
const int GATE_CLOSED_DEG = 0;
const int GATE_OPEN_DEG   = 90;

// in setup():
gate.attach(25);
gate.write(GATE_CLOSED_DEG);

// when the floodgate device reports is_on:
gate.write(on ? GATE_OPEN_DEG : GATE_CLOSED_DEG);

```

8 Battery backup, enclosure and mounting

What turns a breadboard into something that survives a night outdoors — and the one part the buying guide missed.

8.1 You need one part that is not on the buying list

TP4056 PLUS 18650 ALONE CANNOT POWER AN ESP32

An 18650 cell delivers 3.0–4.2 V. The ESP32's `VIN` pin expects 5 V, and its `3V3` pin has an absolute maximum of 3.6 V — so a freshly charged 4.2 V cell wired to `3V3` is over the limit and will eventually damage the board. You need a **boost converter** between them.

Buy either an **MT3608 step-up module** (roughly ₱60, adjust its trimmer to 5.0 V with the multimeter *before* connecting the ESP32), or a combined **18650 power-bank / UPS charging module** (roughly ₱150) that integrates charging, protection and 5 V output on one board. The combined module is the better choice — fewer wires, and it handles charge and discharge at the same time.

Also check which TP4056 variant you received. The bare version has no low-voltage cutoff and will discharge an 18650 until it is permanently ruined. The protected version carries two extra chips beside the charging IC (a DW01 and a dual MOSFET) and disconnects the cell at about 2.5 V. Buy the protected one.

Connection	Goes to
IN+ / IN-	5 V USB adapter — the charging input.
B+ / B-	The 18650 holder. Observe polarity; reversing it destroys the module instantly.
OUT+ / OUT-	The boost converter input. The boost output (set to 5.0 V) then feeds ESP32 <code>VIN</code> and <code>GND</code> .
Battery sense	100 kΩ from <code>B+</code> to a junction, 100 kΩ from that junction to <code>GND</code> , junction to <code>GPIO 34</code> . Halves 4.2 V to 2.1 V, comfortably inside the ADC range.

Once wired, set `REPORT_BATTERY = true` in the firmware, then calibrate: measure the actual cell voltage across `B+ / B-` with the multimeter and compare it against the firmware's computed `volts`. If they disagree, adjust the `2.0f` multiplier in `readBatteryPercent()` by the ratio. Resistors are typically 5 % parts, so a few percent of error here is normal and worth correcting once.

8.2 Enclosure

- **Keep the antenna outside metal.** The IP65 box must be plastic. A metal enclosure will kill Wi-Fi range entirely.
- **Cable glands, not drilled holes.** Every cable entering the box needs a gland, and every gland should enter from the **underside** so water cannot pool at the opening.
- **Drip loop.** Let each cable hang below its entry point before rising into the gland, so water runs off the low point instead of tracking along the cable into the box.
- **The sensor points down through the bottom face**, sealed around the barrel with neutral-cure silicone. Do not seal the transducer face itself.
- **Add a desiccant sachet.** Sealed boxes in a tropical climate condense internally as they cool at night; a silica gel packet costs nothing and keeps the board dry.

8.3 Mounting geometry

Constraint	Value and reason
Height above highest expected water	At least 25 cm — the blind zone. Realistically leave 40 cm so an unexpected surge does not blind the sensor exactly when it matters most.
Angle	Within 5° of vertical.

Clear cone	About 26 cm diameter at 1 m range, widening with distance. No brackets, cables or walls inside it.
Debris clearance	Above the waterline that carries branches and rubbish. A struck sensor is a dead sensor.
Rigidity	Bolt to a bridge underside, culvert headwall or a braced post. Any sway changes the measured distance and appears in the data as phantom water-level noise.

9 Troubleshooting

Symptoms in the order you are likely to meet them.

Symptom	Most likely cause	Fix
Board never appears as a COM port; Port menu greyed out	Charge-only USB cable	First determine whether Windows sees the device at all (see the callout in 1.2). Nothing in Device Manager means the cable, not the driver. Test the cable on a phone — file transfer offered or not.
Device shows a yellow triangle under <code>VID_1A86</code>	WCH driver not installed	Install CH341SER (CH340) or CH343SER (CH9102) as administrator, then replug. Check the PID: <code>7523</code> = CH340, <code>55D4</code> = CH9102.
Driver installed but still no port	Driver staged with nothing to bind to	Confirm with <code>pnputil /enum-drivers</code> that the INF is staged. If it is, the board is not enumerating — go back to the cable.
"Failed to connect to ESP32"	Board not in bootloader mode	Hold BOOT while the IDE prints "Connecting...", release when writing starts.
Board reboots randomly	Insufficient current	Use the 5 V 2 A adapter, not a laptop port. Common once a pump or servo is attached.
Boots to a garbled loop	A strapping pin is held at the wrong level	Nothing should be wired to GPIO 0, 5, 12 or 15. Use the Section 3.1 pin map.
Ultrasonic always "no echo"	Powered from 3V3, or ECHO not reaching the GPIO	VCC must be on <code>VIN</code> . Check the divider midpoint with the multimeter — it should read about 3.3 V during a ping.
JSN-SR04T silent, HC-SR04 fine	v3.0 board in serial mode	Check that the <code>R27</code> pad is unpopulated. A resistor there selects a mode that ignores TRIG.
Distance jumps wildly	Tilt, or an obstruction in the beam cone	Level the sensor to within 5°; clear the 26 cm cone. The median filter handles ripples, not geometry.
Readings stop above a level	Water entered the 25 cm blind zone	Raise the sensor or lower the critical threshold. Recalculate per Section 6.3.
DHT22 always <code>nan</code>	Missing pull-up, or wrong type	Fit the 10 kΩ from DATA to 3V3. If the sensor is blue, it is a DHT11 — change the constructor.
Wi-Fi never connects	5 GHz network	The ESP32 is 2.4 GHz only. Use a 2.4 GHz SSID or a phone hotspot.
<code>HTTP -1</code> from the ESP32	Cannot reach the server	Verify the laptop's LAN IP, that the firewall rule exists, and that both devices share a subnet. Confirm with Gate 5 from another device first.
<code>HTTP 401</code>	Token wrong or missing	Re-issue with <code>php artisan sensors:token {id}</code> and paste the new value into the firmware.
<code>HTTP 404</code>	Sensor ID does not exist	Check Admin → Sensors for the real ID.
<code>HTTP 422</code>	Validation failed	Read the response body — it names the field. Usually a negative <code>water_level</code> from a bad <code>MOUNT_HEIGHT_M</code> .
<code>HTTP 500</code> after adding battery	Controller change from 2.2 not applied	<code>battery_level</code> must be removed from the payload before <code>SensorReading::create()</code> .

Level falls as water rises	Inverted conversion	Re-measure <code>MOUNT_HEIGHT_M</code> per Section 6.1 and re-check the subtraction.
Readings arrive, no alert	Thresholds still at field values	Scale the zone's thresholds to the tank (Section 6.3).
Alert fires once, never again	Working as designed	Alerts are deduplicated per sensor and severity while one is active. Resolve it to allow the next.
"Sensor offline" alerts	Gaps longer than 30 minutes	Confirm the post interval and that failed cycles are not silently repeating — watch the Serial Monitor.
Predictions stay empty	Fewer than 3 readings in 2 hours	Let it run 20 minutes at a 5-minute interval, then <code>php artisan predictions:generate</code> .
Relays on at boot	Active-LOW board	Confirm <code>RELAY_ACTIVE_LOW = true</code> .
Relays never click	3.3 V logic too weak	Remove the <code>VCC-JD-VCC</code> jumper and power <code>JD-VCC</code> from 5 V, grounds common.

10 Acceptance checklist

Tick these in order. Anything unticked is the next thing to work on.

Gate	Result	Done
Server	Migration applied, token issued, API tests updated and passing	☐
1	Serial output and blinking LED	☐
2	Distance tracks a tape measure	☐
3	Temperature and humidity read correctly	☐
4	ESP32 gets an IP on the laptop's subnet	☐
5	<code>curl.exe</code> returns 201 and the reading appears in the dashboard	☐
6	ESP32 posts a fixed reading successfully	☐
7	Real measurement, converted correctly, verified against 6.2	☐
8	Pouring water drives safe → warning → critical with alerts and notifications	☐
9	A prediction with real rate of rise and confidence	☐
Set B	Dashboard toggle switches the physical pump and siren	☐
Set B	Battery level visible in the dashboard; node survives unplugging the adapter	☐
Final	Post interval restored to 5 minutes; all parts in the enclosure	☐

Demonstration sequence

A sequence that tells a complete story in about ten minutes, ending on the strongest point:

1. Show the dashboard at rest, with the real sensor reporting a safe level every five minutes.
2. Pour water steadily. Narrate the level climbing on the live chart.
3. Cross the warning threshold — the alert appears, notifications dispatch to that barangay's residents.
4. Open the notification log to show who was reached and by which channel.
5. Run `predictions:generate` and show rate of rise, time to critical, and confidence derived from the water you just poured.
6. Cross critical. Toggle the pump from the Smart Actuation screen and let the panel watch physical water being moved.

7. Unplug the mains adapter. The node keeps reporting on battery — the point that lands hardest for a flood system in the Philippines.

KEEP THE SIMULATOR AVAILABLE

Do not remove `php artisan sensors:simulate` when the hardware works. One physical node covers one zone; the simulator populates the other six so the map, analytics and zone comparisons look complete. Explain the split openly — one real node proves the data path, simulation proves the platform scales. Panels respect that far more than an unconvincing claim of seven live sensors.